

Hi3519DV500 内存重新分配指南

1. 背景

1.1 内存分配机制

Hi3519DV500 的 DDR 物理内存被分为两部分：

DDR 物理内存（总容量）	
Linux 系统内存 (mem= 参数控制)	MPP 媒体保留内存 (硬件自动保留)
用于： · 进程/线程 · 文件系统缓存 · 网络栈 · 用户态应用 · 内核模块	用于： · VI/VO 视频输入输出 · VENC/VDEC 编解码 · VPSS 图像处理 · ISP 图像信号处理 · AI 推理引擎

- Linux 系统内存：由 bootargs 中的 mem= 参数指定，内核启动后通过 /proc/meminfo 可查
- MPP 媒体保留内存：物理 DDR 总量减去 mem= 的值，由 MPP/ISP 硬件在内核启动前自动保留

核心原则：增大 mem= → Linux 可用内存增加，MPP 媒体缓存减少；减小 mem= → 反之。

1.2 内存分配的关键文件

只有一个文件需要修改：

文件	路径（相对于 bsp/）	作用
emmc_env.txt	tools/pc/uboot_env/env_text/hi3519dv500/emmc_env.txt	u-boot 环境变量文本源，包含 mem= 参数

当前配置：

```
bootargs=mem=262144K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5
rootfstype=ext4 rw rootwait
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

其中 mem=262144K = 256MB，即 Linux 系统当前可见内存为 256MB。

2. 常用内存分配参考值

2.1 常见 DDR 总容量场景

下表以 Hi3519DV500 常见配置为例（实际 MPP 保留量因使用场景而异）：

DDR 总量	mem= (Linux)	MPP 保留	适用场景
256MB	131072K (128MB)	~128MB	基础 IPC，单路 1080P 编码
256MB	196608K (192MB)	~64MB	轻量应用，媒体负载较低
512MB	262144K (256MB)	~256MB	默认配置 ，多路编码 + AI 推理
512MB	393216K (384MB)	~128MB	多应用场景，媒体负载适中
512MB	458752K (448MB)	~64MB	偏 Linux 应用，媒体负载低
1024MB	524288K (512MB)	~512MB	AI + 多通道高分辨率
1024MB	786432K (768MB)	~256MB	多功能网关，多进程
1024MB	917504K (896MB)	~128MB	通用服务器场景

2.2 快速换算

1MB = 1024K	
64MB = 65536K	
128MB = 131072K	
192MB = 196608K	
256MB = 262144K	← 当前默认值
320MB = 327680K	
384MB = 393216K	
448MB = 458752K	
512MB = 524288K	
640MB = 655360K	
768MB = 786432K	
896MB = 917504K	
1024MB = 1048576K	

2.3 MPP 保留内存的最低需求估算

MPP 媒体保留内存取决于实际使用的功能通道。以下为粗略估算：

功能	单通道内存占用（约）
VI 输入 (1080P)	8MB
VENC 编码 (H.264/H.265, 1080P)	8-16MB
VDEC 解码 (1080P)	8-16MB

功能	单通道内存占用（约）
VPSS 图像缩放	4-8MB
VO 输出	4-8MB
AI 推理 (NNIE)	16-64MB (取决于模型)
ISP 缓存	4-8MB

以上为单通道估算，实际需按通道数叠加，并预留 20%-30% 余量。

3. 修改步骤

提供两种方案：**方案 A（编译烧录）** 适用于批量生产和版本管理，**方案 B（u-boot 命令行）** 适用于开发调试和单台设备快速验证。

3.1 方案 A：修改 emmc_env.txt + 编译烧录

步骤 1：修改 emmc_env.txt

文件路径：`bsp/tools/pc/u-boot_env/env_text/hi3519dv500/emmc_env.txt`

找到 `bootargs=` 行，修改其中的 `mem=` 值。只改 `mem=` 后面的数字，不要动其他参数。

示例——将 Linux 内存从 256MB 增加到 384MB：

```
# 修改前
bootargs=mem=262144K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5
rootfstype=ext4 rw rootwait
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)

# 修改后
bootargs=mem=393216K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5
rootfstype=ext4 rw rootwait
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

步骤 2：重新编译

```
cd /workspace/.../bsp
make u-boot_env
```

该命令会同时生成两个产物：

- `emmc_env.bin` — 烧写到 eMMC env 分区，u-boot 启动时读取
- `emmc_burn_table.xml` — 烧写分区表（修改 `mem=` 不影响分区表，但该文件仍会重新生成）

产物输出位置：

```
pub/hi3519dv500_emmc_image_musl/
├─ emmc_env.bin          ← 重新生成
└─ emmc_burn_table.xml   ← 重新生成（内容不变）
```

步骤 3：重新烧录

将重新生成的 `emmc_env.bin` 烧写到板子的 eMMC env 分区。

方式一：使用烧写工具全量烧录

将 `pub/hi3519dv500_emmc_image_musl/` 下的所有镜像文件一起烧写。

方式二：仅更新 env 分区（推荐，速度快）

在 u-boot 命令行中通过 TFTP/串口单独更新 env 分区。具体方法取决于所使用的烧写工具。

3.2 方案 B：u-boot 命令行直接修改（无需编译烧录）

此方案通过板子串口终端进入 u-boot 命令行，直接修改 `bootargs` 环境变量，**不需要开发机编译、不需要烧录工具**。

进入 u-boot 命令行

1. 通过串口连接板子（波特率 115200）
2. 板子上电或复位，在 u-boot 倒计时（默认 2 秒）内按任意键打断启动
3. 进入 `=>` 提示符

步骤 1：查看当前 bootargs

```
=> printenv bootargs
bootargs=mem=262144K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5
rootfstype=ext4 rw rootwait
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

步骤 2：设置新的 bootargs

使用 `setenv` 命令修改 `bootargs`，**必须保留完整的一行参数，只改 `mem=` 的值**：

```
=> setenv bootargs mem=393216K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5
rootfstype=ext4 rw rootwait
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

步骤 3：保存或直接启动

根据需要选择以下方式之一：

临时生效（仅本次启动，重启后恢复原值）：

```
=> boot
```

适合快速试验某个 `mem=` 值是否可行。如果启动异常，直接复位板子即可恢复。

永久生效（写入 eMMC env 分区，重启后保留）：

```
=> saveenv
Saving Environment to MMC...
Writing to MMC(0)... done

=> boot
```

`saveenv` 会将当前环境变量写入 eMMC 的 env 分区，后续每次启动都会使用新值。

步骤 4：恢复默认值（可选）

如果需要恢复到编译时的原始值：

```
=> env default bootargs
=> saveenv
```

3.3 两种方案对比

	方案 A：编译烧录	方案 B：u-boot 命令行
操作位置	开发机上修改文件	板子串口终端
是否需要编译	需要 <code>make uboot_env</code>	不需要
是否需要烧录工具	需要	不需要
适用场景	批量生产、版本管理、团队协作	开发调试、单台设备快速验证
可追溯性	有 git 记录	无记录，需人工备注
风险	低，有源文件可回溯	中， <code>setenv</code> 拼写错误可能导致启动失败
恢复方式	还原文件重新编译烧录	<code>env default bootargs</code> + <code>saveenv</code>

推荐 workflow：先用方案 B 在单块板子上快速试验，找到合适的 `mem=` 值后，再用方案 A 将确定值写入 `emmc_env.txt` 统一管理。

4. 验证

4.1 编译验证

```
# 确认产物更新时间
ls -la pub/hi3519dv500_emmc_image_musl/emmc_env.bin
```

4.2 u-boot 阶段验证

烧写后重启，在 u-boot 倒计时按任意键进入命令行：

```
=> printenv bootargs
```

确认输出中 `mem=` 值为修改后的值。

4.3 Linux 内核阶段验证

内核启动完成后：

```
# 查看 Linux 可用内存总量
cat /proc/meminfo | head -3

# MemTotal 应该约等于 mem= 设置的值（略小几十 KB 是正常的内核开销）
# 例如设置 mem=393216K，MemTotal 应在 383000 KB 左右

# 查看实际内存分布
cat /proc/iomem | grep "System RAM"
```

4.4 MPP 媒体功能验证

修改内存后**必须**验证 MPP 媒体功能是否正常：

```
# 加载 MPP 模块并启动 VI/VENC 通道
# 具体命令取决于您的 MPP 初始化脚本

# 检查 MPP 内核日志有无内存分配失败
dmesg | grep -i "alloc\|fail\|mpp\|vb"

# 如果出现 VB (Video Buffer) 分配失败，说明 MPP 保留内存不足
# 需要减小 mem= 值，将更多内存还给 MPP
```

5. 常见问题

Q1: 修改 mem= 后内核启动崩溃或卡死

原因： `mem=` 值超过了物理 DDR 扣除硬件保留后的可用范围。

解决：

1. 先在 u-boot 命令行确认实际 DDR 大小： `=> bdinfo`
2. 从小值开始逐步增大，比如先设 `mem=131072K`，确认能启动后再逐步增加

Q2: 内核正常启动但 MPP 功能异常 (VENC/VI 报错)

原因：MPP 保留内存不足，VB (Video Buffer) 池无法分配足够缓存。

解决：减小 `mem=` 值，为 MPP 保留更多内存。参考第 2.3 节估算 MPP 最低需求。

Q3: 如何确定最佳 mem= 值

方法：逐步逼近法。

- 1. 从保守值开始 (DDR 总量的 50%)
- 2. 启动系统，验证 MPP 功能正常
- 3. 逐步增大 `mem=` (每次增加 32MB = 32768K)
- 4. 在每一步都验证 MPP 功能
- 5. 直到出现 MPP 异常，回退到上一个正常值

Q4: mem= 单位必须是 K 吗

不一定。Linux 内核 `mem=` 支持以下单位：

单位	含义	示例
K	KiB	<code>mem=262144K</code>
M	MiB	<code>mem=256M</code>
G	GiB	<code>mem=1G</code>

SDK 中统一使用 `K` 单位，精度更高。建议保持一致使用 `K`。

Q5: 修改 mem= 需要重新编译内核或 u-boot 吗

不需要。`mem=` 是通过 u-boot 环境变量传递给内核的启动参数，只修改 `emmc_env.txt` 并重新 `make uboot_env` 即可。

Q6: 多颗芯片的内存分配不同怎么办

每颗芯片的 `emmc_env.txt` 是独立的。如需为不同板子配置不同内存，可以：

- 修改各自的 `emmc_env.txt` 文件，分别编译烧录
- 或者通过方案 B (u-boot 命令行) 在单板上单独设置，见第 3.2 节

Q7: u-boot setenv 拼写错误导致无法启动怎么办

如果 `setenv bootargs` 时误删了关键参数 (如 `root=`、`rootfstype=`)，导致内核启动失败：

- 1. 复位板子，在 u-boot 倒计时按任意键进入命令行
- 2. 恢复默认值：`=> env default bootargs` 然后 `=> saveenv`
- 3. 如果 `saveenv` 也无法恢复 (env 分区损坏)，需重新烧录 `emmc_env.bin`

建议：在 `setenv` 之前先执行 `printenv bootargs` 记录当前完整值，以便回退。

6. 注意事项

1. **mem= 值不能超过物理 DDR 总量。**超出会导致内核启动崩溃或内存越界访问
2. **必须为 MPP 预留足够内存。**如果计划使用视频编解码、AI 推理等功能，MPP 至少需要 64MB 以上保留内存
3. **修改后必须重新烧录** `emmc_env.bin`，仅编译不会影响已运行的设备
4. **建议先在开发板上验证**，确认功能正常后再批量烧录
5. **生产环境建议保留 MPP 内存的 20%-30% 余量**，避免极端场景下 VB 分配失败